Credit Risk Scoring & Portfolio Optimization

Technical Methodology & Architecture

End-to-end pipeline for multi-segment credit decisioning

with MILP optimization, reject inference, and automated reporting

February 2026 | Confidential

Confidential

Agenda

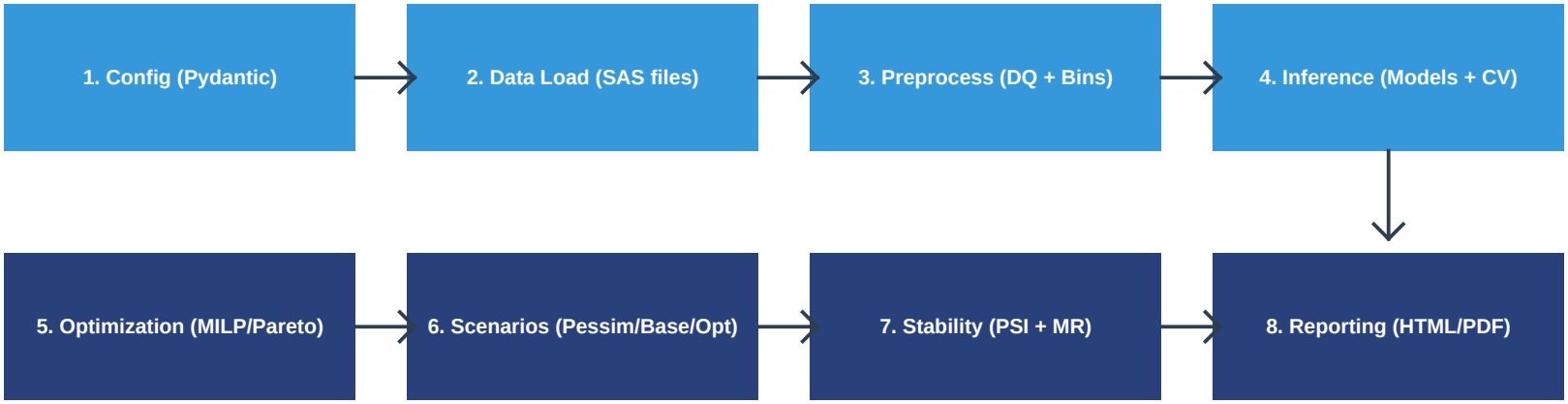
Presentation outline

1. Pipeline Architecture & End-to-End Flow
2. Configuration System (Pydantic, N-Variable)
3. Preprocessing & Binning Methodology
4. Model Inference (Hurdle, Tweedie, CV)
5. Optimization (MILP, Pareto, Monotonicity)
6. Reject Inference & Selection Bias
7. Scenario Analysis & Bootstrap CI
8. Stability (PSI/CSI) & MR Validation
9. Trend Analysis & SPC Anomaly Detection
10. Batch Processing & Global Allocation
11. Consolidated Reporting (Redesigned)
12. Recent Improvements & Roadmap

Confidential

Pipeline Architecture

End-to-end flow from raw data to decisions



Optional Pipeline Extensions

- ✔ **6b.** Sensitivity Analysis: Risk parameter perturbation across grid
- ✔ **6c.** Reject Inference Optimization: Grid search over RI parameters
- ✔ **6d.** Re-optimization: Rerun Step 5 with tuned RI parameters

Key Design Principles

- ✔ **Path Management:** Centralized OutputPaths dataclass for all file I/O
- ✔ **Error Isolation:** Non-blocking optional steps (trends, reporting, sensitivity)
- ✔ **Typed Errors:** 3 custom exceptions: ConfigLoadError, DataLoadError, InferencePhaseError
- ✔ **Early Exit:** Training-only mode, pre-trained model loading, skip DQ flags

Confidential

Configuration System

Pydantic-validated, N-variable, two-tier config

Two-Tier Configuration

config.toml provides global defaults. segments.toml overrides per-segment. All settings flow through PreprocessingSettings (Pydantic BaseModel) with field and model validators ensuring correctness at load time.

PreprocessingSettings - Key Parameters

Parameter	Type	Default	Description
variables	list[str]	(required, >=2)	Score variable names for the grid
bins	Dict[str, BinConfig]	(auto from legacy)	N-variable binning config
inference_variables	list[str]	= variables	Feature subset for model training
optimum_risk / risk_step	float	1.1 / 0.1	Risk target % and scenario increment
multiplier	float	7.0	b2_ever_h6 risk multiplier
reject_inference_method	str	none	none / parceling

BinConfig (per variable)

```
source_col: str        # Raw column (e.g., 'score_rf')
output_col: str        # Binned column (e.g., 'sc_octroi_new_clus')
bin_edges: list[float] # Thresholds (>= 2 edges required)
method: str            # 'quantile' | 'optimization' | 'supervised'
```


Confidential

Preprocessing & Binning

Data quality, filtering, and discretization

Data Quality & Filtering Pipeline

- ✔ Filter: fuera_norma='n', fraud_flag='n', nature_holder != 'legal'
- ✔ Date range filtering with monthly distribution logging
- ✔ Status/reject-reason reclassification based on direct measures
- ✔ Outlier flagging via z-score (configurable z_threshold, default 3.0)

Binning Methods

Method	Algorithm	Edge Output	Best For
quantile	Equal-count quantile splits	[-inf, q1, q2, ..., inf]	Unsupervised, avoids leakage
optimization	DecisionTreeRegressor + production weig	[-inf, t1, t2, ..., inf]	Production-weighted risk split
supervised	DecisionTreeClassifier max_leaf_nodes	[-inf, t1, t2, ..., inf]	Legacy binary classification

Monotonicity Direction Inference

Uses weighted Spearman correlation on booked data (weighted by production). Direction = 1: higher bin = riskier (ascending risk). Direction = -1: higher bin = safer (descending risk). Statistical significance via permutation test (1000 reps, p > 0.10 threshold).

Gini Assessment

Compares AUC-based Gini for raw score vs. binned score to quantify information retention. Reports gini_raw, gini_binned, and gini_retention_%.

Confidential

Model Inference

Custom estimators, feature engineering, cross-validation

HurdleRegressor (Two-Stage Zero-Inflated Model)

Stage 1: Classification

LogisticRegression(max_iter=1000)
Predicts P(Y > 0) vs P(Y = 0)
Separates zero from non-zero outcomes

Stage 2: Regression

Ridge(alpha=0.5)
Predicts E[Y | Y > 0] on non-zero subset
Final: P(Y>0) x E[Y|Y>0]

TweedieGLM (Generalized Linear Model)

Tweedie distribution (power=1.5, alpha=0.5, link='log') for compound Poisson-Gamma. Supports exposure offset: internally converts ratio targets to numerator scale, fits on log(exposure) as feature, returns predictions on ratio scale.

Feature Engineering & Selection

Feature Set	Variables (2-var)	Variables (N-var)
original / degree_1	[var0, var1]	Raw variables
base / degree_2	[var0, var1, var0*var1]	Degree-2 polynomial
base + squared	base + [var0^2, var1^2]	Degree-2 polynomial
full / degree_3	All up to cubic + interactions	Degree-3 polynomial

One-Standard-Error Rule

Among models within 1 SE of best CV RMSE, select the simplest. Prevents overfitting from unnecessarily complex feature sets. K-Fold CV (default k=4), RMSE weighted by group size.

Confidential

Optimization

MILP formulation with monotonicity constraints

CellGrid: N-Dimensional Abstraction

CellGrid dataclass maps N score variables to a flat cell index. Each cell aggregates KPIs (production, defaults, exposure, count). Supports 2D legacy (octroi x efx) and N-D modern grids (3+ variables).

Mixed-Integer Linear Program (MILP)

```
Objective:   max  SUM( oa_amt_h0[i] * x[i] )

Risk budget: SUM( (mult * todu_30ever_h6[i]
                   - target_risk/100 * todu_amt_pile_h6[i]) * x[i] ) <= 0

Monotonicity: x[riskier_cell] - x[safer_cell] <= 0
              for all adjacent pairs along each dimension

Integrality:  x[i] in {0, 1}   (accept/reject per cell)
```

Monotonicity Constraints

Ensures that if a riskier cell is rejected, all cells riskier along that dimension are also rejected. Built as sparse CSC matrix for ~100x faster solving. Direction per variable from inference or config override.

Solver & Fallback

- ✔ Primary: scipy.optimize.milp with 30-second timeout per risk target
- ✔ Fallback: pymoo Genetic Algorithm (pop=100, gen=50) if MILP finds no feasible solution

Confidential

Pareto Frontier

Risk vs. production trade-off analysis

Frontier Construction Algorithm

- ✔ **Step 1:** Risk sweep: 50 linearly-spaced targets from 0.01% to max_risk * 1.1%
- ✔ **Step 2:** MILP solve for each target, yielding (risk, production, mask) tuples
- ✔ **Step 3:** Deduplicate: track unique binary masks to avoid redundant solutions
- ✔ **Step 4:** Sort by ascending risk, filter to strictly increasing production
- ✔ **Step 5:** Post-hoc dominance verification: remove any remaining dominated points

Scenario Selection on the Frontier

Scenario	Risk Target	Interpretation
Pessimistic	optimum_risk - risk_step	Conservative: lower risk tolerance
Base	optimum_risk	Central estimate: target risk level
Optimistic	optimum_risk + risk_step	Aggressive: higher risk tolerance

Each scenario selects the Pareto-optimal solution closest to its risk target. Bootstrap CI (standard error) computed per scenario for risk and production. Output includes accepted/rejected cell counts, swap-in/swap-out analysis.

Output Artifacts per Scenario

- ✔ risk_production_summary_table: Actual vs Optimum vs Swap metrics
- ✔ optimal_solution: Per-variable cutoff values + KPIs
- ✔ cutoff_summary_wide + acceptance_grid: Full grid with accept/reject and Plotly heatmap

Confidential

Reject Inference

Correcting selection bias in observed outcomes

The Problem

We only observe default outcomes for booked (accepted) applications. Rejected applications are unobserved, creating selection bias. Without correction, risk estimates are biased downward for cells with low acceptance rates.

Methods

Method	Formula	Behavior
none	No adjustment	Conservative: ignores rejected population
parceling (linear)	mult = 1 + factor * (1 - AR)	Mild: linear increase as AR drops
parceling (power)	mult = (1/AR) ^ factor	Stronger: exponential for low AR

Algorithm Detail

- ✔ **Step 1:** Acceptance rate AR[bin] = n_booked / (n_booked + n_score_rejected)
- ✔ **Step 2:** Compute risk multiplier per bin using selected formula
- ✔ **Step 3:** Cap multiplier at max_risk_multiplier (default 3.0)
- ✔ **Step 4:** Adjust: todu_30ever_h6_adjusted = todu_30ever_h6 * multiplier

Only risk is adjusted (production is observable for all applications). Parameters (factor, max_multiplier) can be auto-tuned via grid search in Step 6c.

Confidential

Stability & MR Validation

PSI/CSI monitoring and out-of-time validation

Population Stability Index (PSI)

```
PSI = SUM( (Actual% - Expected%) * ln(Actual% / Expected%) )
```

PSI Range	Interpretation	Action
< 0.10	Stable	No action needed
0.10 - 0.25	Moderate shift	Monitor closely
>= 0.25	Significant shift	Investigate & recalibrate

Implementation: bins from baseline quantiles, epsilon=0.0001 for zero protection. CSI extends to multivariate case (per-variable PSI). Drift alerts stored as JSON for automated monitoring.

Marginal Risk (MR) Validation

- ✔ Load separate MR-period data (more recent observation window)
- ✔ Apply optimal cutoffs from main period to MR data
- ✔ Compare booked-in-main vs booked-in-MR risk per bin
- ✔ Hybrid: use MR outcomes if >= min_obs_per_bin, else fallback to main period
- ✔ Output: risk/production summary with Actual / Swap-in / Swap-out breakdown

Confidential

Trend Analysis & Anomaly Detection

Statistical Process Control on monthly metrics

Monthly Metric Aggregation

- ✔ Metrics computed per year-month: approval_rate, total_records, booked_count
- ✔ Production metrics: mean_production, total_production
- ✔ Risk metrics: risk_rate per score bin
- ✔ Score distribution: mean_sc_octroi, mean_new_efx, mean_risk_score_rf

SPC Anomaly Detection

```
Rolling mean:  mu_t   = mean(x_{t-w}, ..., x_{t-1})   [window=6]
Rolling std:   sigma_t = std(x_{t-w}, ..., x_{t-1})

Anomaly flag:  |x_t - mu_t| > n_sigma * sigma_t       [n_sigma=2.0]
Direction:     'up' if x_t > mu_t else 'down'
```

Output: DataFrame with control bounds (upper/lower), flags, direction. Interactive Plotly visualization with trend line (degree-1 polynomial) per metric.

Confidential

Batch Processing & Global Allocation

Multi-segment execution and portfolio-level MILP

Batch Runner (run_batch.py)

- ✔ Two-tier config: base config.toml + per-segment overrides in segments.toml
- ✔ Global bin learning: learn edges once on full unfiltered population
- ✔ Parallel execution via ProcessPoolExecutor (configurable workers)
- ✔ Per-segment loggers to prevent cross-talk in parallel mode
- ✔ Supersegment training: train on combined data, apply to individual segments

```
python run_batch.py                          # All segments
python run_batch.py -s seg1 seg2             # Specific
python run_batch.py --parallel --workers 4   # Parallel
python run_batch.py --global-bin-learning    # Learn bins globally
```

Global Allocation (Portfolio Optimization)

Given N segment Pareto frontiers, select one operating point per segment to maximize global production subject to a global risk constraint.

Method	Algorithm	Complexity
Exact (MILP)	Binary vars per (segment, solution) pair	Optimal but slower
Greedy	Iteratively add highest-production point	Fast approximation

Confidential

Consolidated Reporting

Redesigned cutoff comparison (recent improvement)

Problem with Previous Design

The old Cutoff Comparison section dumped each segment's cutoff_summary_wide.csv as a raw flat table (capped at 200 of ~1,200 rows). Portfolio KPIs repeated on every row, grid cells truncated. Result: unreadable and full of redundant data.

New Design: Two Focused Sub-Sections

1. Scenario KPI Summary Table

One row per (segment x scenario) = ~9 rows.
Columns: Risk%, Production(M), CIs,
Accepted/Total counts, Acceptance Rate.
Sorted: pessimistic -> base -> optimistic.

2. Acceptance Matrices (Base)

Colored HTML pivot grids per segment.
Green = accepted, Red = rejected, Grey = N/A.
Supports N-variable slicing (sub-tables).
Max 12 slice combinations displayed.

Implementation: 6 New Helper Functions

- ✔ _detect_variable_cols(): Identify grid variables vs fixed KPI columns
- ✔ _read_cutoff_data(): Concatenate CSVs across segments, handle missing files
- ✔ _build_scenario_kpi_table(): Format production in millions, CIs as [lo, hi]
- ✔ _build_acceptance_matrices(): Pivot + color-code accept/reject per cell
- ✔ _render_acceptance_pivot(): Inline-styled HTML with green/red/grey cells
- ✔ _build_cutoff_comparison_section(): Orchestrate KPI table + matrices

Confidential

Recent Improvements

Latest enhancements to the pipeline

N-Variable Grid Support

Extended from 2D (octroi x efx) to arbitrary N-dimensional grids. CellGrid abstracts dimensionality. Monotonicity constraints generalized to N-D. BinConfig per variable with independent method selection.

Standard Error for Confidence Intervals

Bootstrap standard error replaces simple percentile CI. One-SE rule for model selection: among models within 1 SE of best, pick simplest. NaN handling improved in sensitivity analysis.

Risk Multiplier Integration

Dynamic per-segment risk multiplier (configurable, default 7.0). Propagated through optimization, scenario analysis, and reporting. Allows segment-specific risk scaling based on business rules.

Hybrid MR Risk

When MR-period data has sufficient observations per bin (>= mr_min_obs_per_bin), use actual MR outcomes. Otherwise fallback to main period estimates. Combines out-of-time validation accuracy with main period stability.

Reject Inference Parameter Optimization

Automated grid search over (uplift_factor, max_risk_multiplier) space. Objective: maximize production subject to risk constraint on re-optimized frontier. Eliminates manual RI parameter tuning.

Confidential

Testing & Code Quality

Comprehensive test suite and CI/CD

Test Suite

Metric	Value
Total Tests	630
Coverage Target	80% on patch (codecov)
Test Framework	pytest with synthetic DataFrames
Reproducibility	numpy.random.RandomState(42)
CI Pipeline	lint -> test -> Docker build (push/PR to main)

Code Quality Tools

- ✔ Ruff: lint (E, F, W, I, UP, B, SIM, N) + format, line length 120
- ✔ Type safety: Pydantic validators, Enum constants (StatusName, RejectReason, Columns)
- ✔ Conditional imports: skip guards for optional deps (shap, pandera)

Deployment

- ✔ Docker: single-stage build, uv-based install
- ✔ Entry points: main.py (single), run_batch.py (multi), run_allocation.py (MILP)
- ✔ Web dashboards: dashboard.py / interactive_allocator.py (Dash)

Confidential

Summary

A complete, production-ready credit decisioning pipeline

- ✔ Multi-segment batch processing with parallel execution
- ✔ N-dimensional score grids with configurable binning methods
- ✔ Custom ML estimators (Hurdle, Tweedie) with CV feature selection
- ✔ MILP optimization with monotonicity constraints on Pareto frontier
- ✔ Reject inference with automated parameter tuning
- ✔ Bootstrap CI, PSI/CSI stability, MR validation
- ✔ SPC trend monitoring with anomaly detection
- ✔ Redesigned consolidated reporting with acceptance matrices
- ✔ 630 tests, Pydantic config, Docker-ready deployment

Confidential